

## Part 1:

### 1.1

```
appdb=# \dt
```

```
      List of tables
 Schema | Name | Type  | Owner
-----+-----+-----+-----
 public | hero | table | postgres
 public | team | table | postgres
(2 rows)
```

```
appdb=# \d hero
```

```
      Table "public.hero"
  Column      |      Type       | Collation | Nullable |      Default
-----+-----+-----+-----+-----
 id           | integer         |           | not null | nextval('hero_id_seq'::regclass)
 name        | character varying |           | not null |
 age         | integer         |           |          |
 team_id     | integer         |           |          |
```

Indexes:

"hero\_pkey" PRIMARY KEY, btree (id)

Foreign-key constraints:

"hero\_team\_id\_fkey" FOREIGN KEY (team\_id) REFERENCES team(id)

### 1.2

Query

Query History

```
-- 1. All heroes aged 18 or older, oldest first.
select * from hero where age >=18 order by age desc;
```

Data Output

Messages

Notifications

Showing rows: 1 to 3

Page No: 1 of 1

|   | id<br>[PK] integer | name<br>character varying | age<br>integer | team_id<br>integer |
|---|--------------------|---------------------------|----------------|--------------------|
| 1 | 3                  | Logan                     | 150            | 2                  |
| 2 | 1                  | Tony                      | 45             | 1                  |
| 3 | 2                  | Natasha                   | 35             | 1                  |

✓ Successfully run. Total query runtime: 147 msec. 3 rows affected. ✕

Total rows: 3

Query complete 00:00:00.147

CRLF

Ln 39, Col 53

Query

Query History

Execute query  
Alt F5

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

--2. Each hero's name next to their team's name  
select h.name as hero\_name, t.name as team\_name  
from hero h  
left join team t on h.team\_id=t.id;

Data Output

Messages

Notifications

Showing rows: 1 to 4

Page No: 1 of 1

|   | hero_name<br>character varying | team_name<br>character varying |
|---|--------------------------------|--------------------------------|
| 1 | Tony                           | Avengers                       |
| 2 | Natasha                        | Avengers                       |
| 3 | Logan                          | X-Men                          |
| 4 | Peter                          | [null]                         |

✓ Successfully run. Total query runtime: 111 msec. 4 rows affected. ✕

Total rows: 4    Query complete 00:00:00.111    CRLF    Ln 52, Col 1

Windows icons

2:50 PM  
10/1/2026

Query

Query History

Execute query  
Alt F5

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

--3. The number of heroes per team.  
select t.name as team\_name, count(h.id) as hero\_count  
from team t  
left join hero h on t.id=h.team\_id  
group by t.id, t.name;

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

|   | team_name<br>character varying | hero_count<br>bigint |
|---|--------------------------------|----------------------|
| 1 | X-Men                          | 1                    |
| 2 | Avengers                       | 2                    |

✓ Successfully run. Total query runtime: 161 msec. 2 rows affected. ✕

Total rows: 2    Query complete 00:00:00.161    CRLF    Ln 67, Col 23

Windows icons

2:50 PM  
10/1/2026

```

80  --4. The 2nd page of heroes when a page has 2 rows, ordered by id.
81  select * from hero
82  order by id
83  limit 2 offset 2;

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

|   | id<br>[PK] integer | name<br>character varying | age<br>integer | team_id<br>integer |
|---|--------------------|---------------------------|----------------|--------------------|
| 1 | 3                  | Logan                     | 150            | 2                  |
| 2 | 4                  | Peter                     | 16             | [null]             |

✓ Successfully run. Total query runtime: 174 msec. 2 rows affected. ✕

Total rows: 2 Query complete 00:00:00.174 CRLF Ln 83, Col 18

1.3

**Question 1:** UNIQUE, 'Avengers' da ton tai trong bang team

FK, team\_id =99 khong ton tai NOT NULL, khong truyen gia tri cho cot name

FK, cac dong trong bang hero danh tham chieu toi team\_id=1 nen chua xoa duoc

**Question 2:** 1 team chua nhieu hero, 1 hero thuoc ve 1 team nen FK team\_id phai nam o bang hero.

**Question 3:** link table

hero: id(PK), name, age

mission: id(PK), title

hero\_mission: hero\_id(FK), mission\_id(FK), PK(hero\_id, mission\_id)

1.4

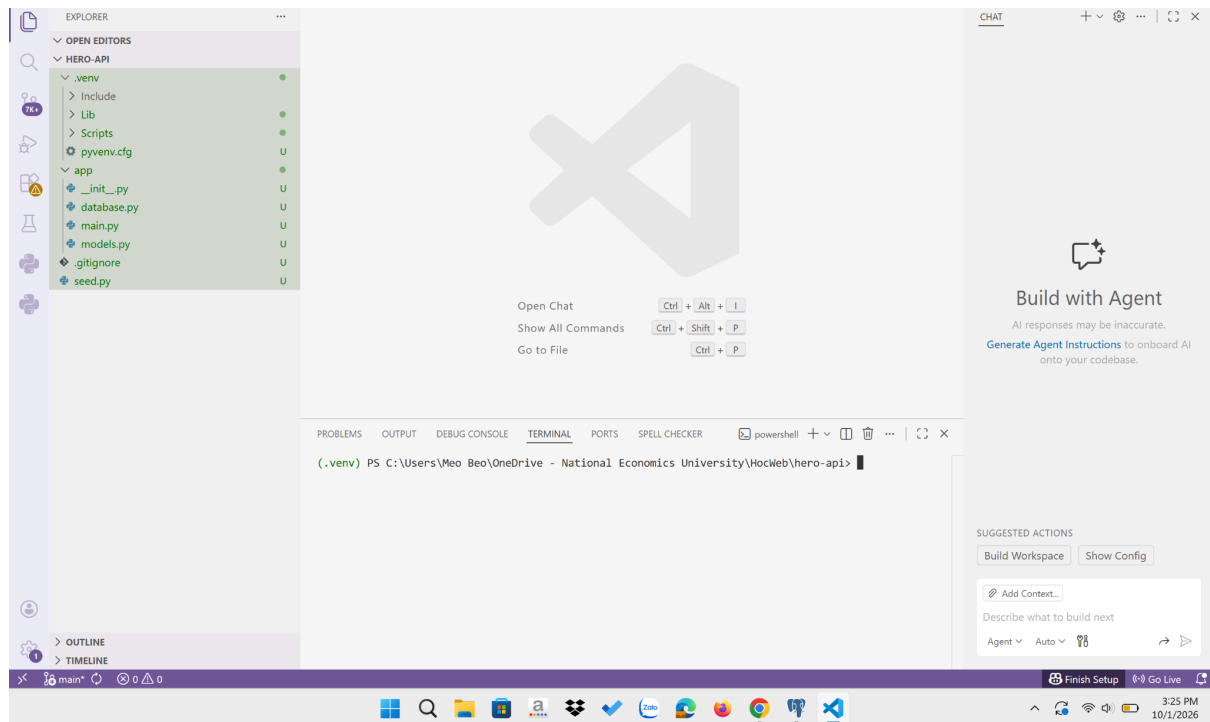
```

appdb-# drop\dt
Did not find any tables.
appdb-# █

```

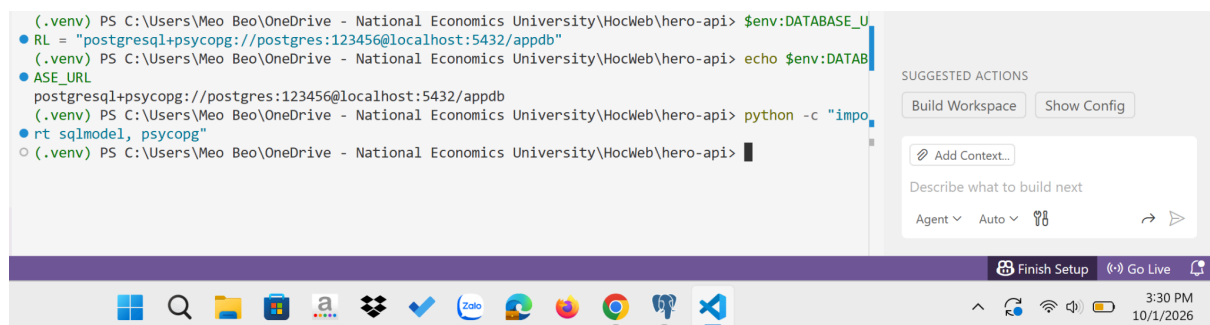
PART 2:

## 2.1



## 2.2

## 2.3



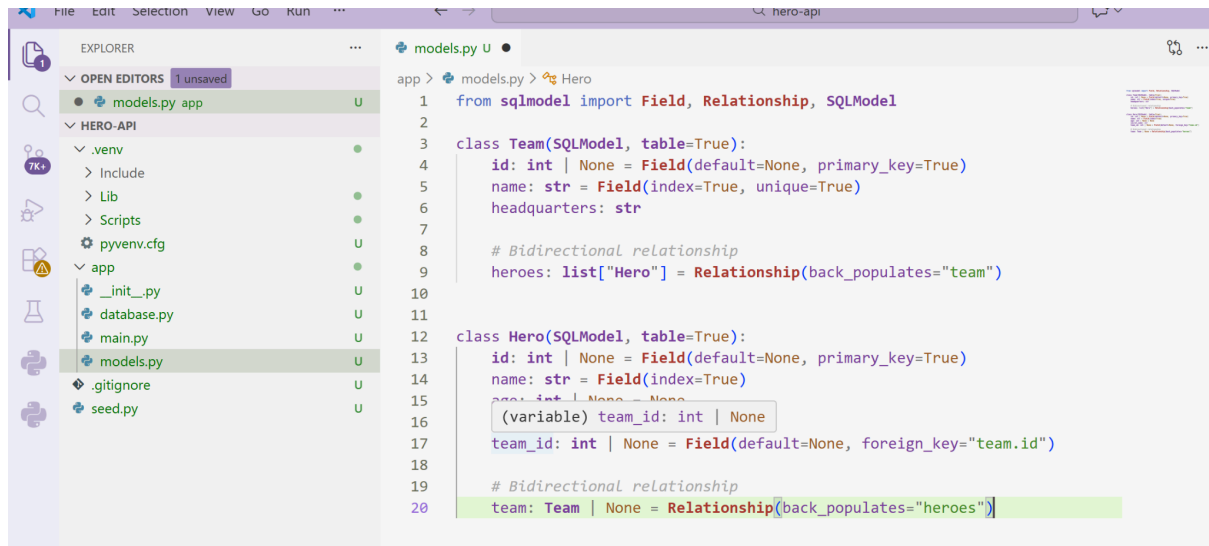
## Question 4:

**Bảo mật :** Tránh làm rò rỉ các thông tin nhạy cảm lên các hệ thống quản lý mã nguồn công khai như GitHub/GitLab.

**Linh hoạt theo môi trường:** Dễ dàng thay đổi cấu hình kết nối khi chạy ở các môi trường khác nhau mà không cần sửa đổi mã nguồn ứng dụng.

## PART 3:

## 3.1



```
1 from sqlalchemy import Field, Relationship, SQLModel
2
3 class Team(SQLModel, table=True):
4     id: int | None = Field(default=None, primary_key=True)
5     name: str = Field(index=True, unique=True)
6     headquarters: str
7
8     # Bidirectional relationship
9     heroes: list["Hero"] = Relationship(back_populates="team")
10
11
12 class Hero(SQLModel, table=True):
13     id: int | None = Field(default=None, primary_key=True)
14     name: str = Field(index=True)
15     age: int | None = None
16     (variable) team_id: int | None
17     team_id: int | None = Field(default=None, foreign_key="team.id")
18
19     # Bidirectional relationship
20     team: Team | None = Relationship(back_populates="heroes")
```

**Question 5:** Vì khi khởi tạo một đối tượng (model) mới trong Python trước khi lưu vào Database, **id** chưa được cơ sở dữ liệu sinh ra nên giá trị tạm thời của nó là **None**. Chỉ sau khi lưu (**session.commit()**), Database mới tự động gán giá trị **id** (**int**) cho đối tượng đó.

**Question 6:**

**Cột trong DB:** **id**, **name**, **age**, **secret\_name**, **team\_id**.

**Không phải cột trong DB:** **team** (đây chỉ là thuộc tính quan hệ do SQLAlchemy quản lý trên Python, không sinh ra cột mới trong bảng **hero** của Database).

**Tác dụng của **back\_populates**:** Dùng để liên kết quan hệ hai chiều (bidirectional relationship). Nó thông báo cho SQLAlchemy biết khi một thuộc tính bên phía **Team** (**heroes**) thay đổi thì thuộc tính tương ứng bên phía **Hero** (**team**) cũng sẽ tự động đồng bộ theo và ngược lại

## 3.2

The screenshot shows a Visual Studio Code editor window with a Python file named `models.py` open. The code defines a `TeamBase` class and a `Team` class using SQLAlchemy. The `Team` class has a `heroes` relationship with a `Hero` model. The `TeamCreate` class is also defined. The terminal shows the command `python -c "from app.models import Hero, HeroPublic; print(Hero.__tablename__)"` being executed, which outputs `hero`.

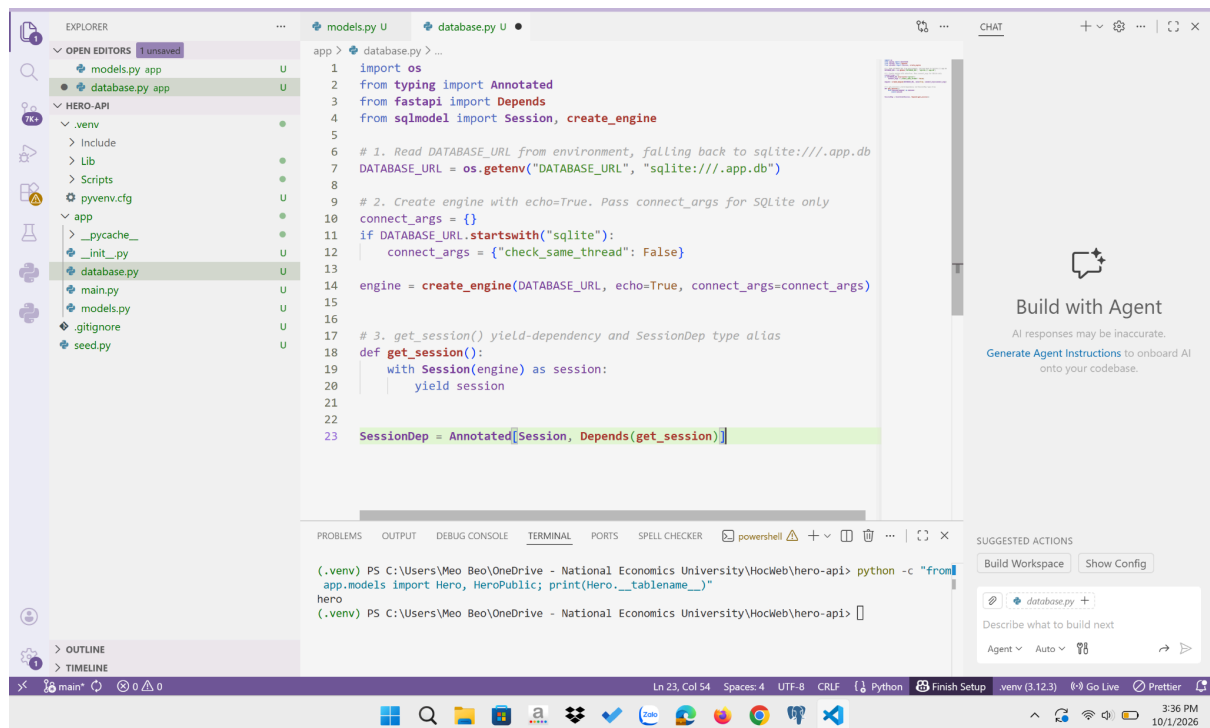
On the right side of the editor, there is a sidebar with the following content:

- Build with Agent**: A section with a speech bubble icon and the text "AI responses may be inaccurate. Generate Agent Instructions to onboard AI onto your codebase."
- SUGGESTED ACTIONS**: A section with buttons for "Build Workspace" and "Show Config". Below these is a search bar with the text "Describe what to build next" and a dropdown menu with "Agent" and "Auto" options.

The bottom status bar shows the file is at line 57, column 35, with 4 spaces, UTF-8 encoding, and CRLF line endings. It also shows the Python interpreter is set to `.venv (3.12.3)` and the Prettier extension is installed.

## PART 4:

### 4.1



## 4.2

### Question 7:

**Kiểu dữ liệu & Ràng buộc:** SQLAlchemy dùng kiểu **VARCHAR** có kèm độ dài tối đa hoặc mặc định phù hợp, quản lý các thuộc tính **NOT NULL** chính xác theo kiểu khai báo Python (ví dụ: **secret\_name** là **NOT NULL**).

**Indexes:** SQLAlchemy tự động tạo câu lệnh **CREATE INDEX** riêng cho các trường có khai báo **index=True** (như cột **name**).

**Question 8:** Không. Vì **create\_all()** sẽ kiểm tra sự tồn tại của các bảng trong cơ sở dữ liệu trước, nếu bảng đã tồn tại rồi thì nó sẽ bỏ qua và không tạo lại.

**Question 9:** Dòng **from app import models** (hoặc **import app.models**). Việc import này thực thi file **models.py** và đăng ký các lớp kế thừa từ **SQLModel** vào **SQLModel.metadata**.

```
appdb-# \dt
          List of tables
Schema | Name | Type | Owner
-----+-----+-----+-----
public | hero | table | postgres
public | team | table | postgres
(2 rows)

appdb-#
```





5.1

Responses

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/heroes' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "name": "string",
  "age": 0,
  "team_id": 0,
  "secret_name": "string"
}'
```

Request URL

```
http://127.0.0.1:8000/heroes
```

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/heroes?offset=0&limit=10' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/heroes?offset=0&limit=10
```

Server response

Code

Details

200

Response body

```
[
  {
    "name": "string",
    "age": 0,
    "team_id": 0,
    "id": 1
  },
  {
    "name": "string",
    "age": 0,
    "team_id": 0,
    "id": 2
  }
]
```

Response headers

```
content-length: 91
content-type: application/json
date: Fri, 02 Oct 2026 02:21:37 GMT
server: uvicorn
```

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/heroes/1' \
  -H 'accept: */*' \
```

Request URL

http://127.0.0.1:8000/heroes/1

Server response

| Code | Details                                                                                                                                                                                                                                |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "name": "string",   "age": 0,   "team_id": 0,   "id": 1 }</pre> <p>Response headers</p> <pre>content-length: 44 content-type: application/json date: Fri, 02 Oct 2026 02:22:16 GMT server: uvicorn</pre> |

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/heroes/2' \
  -H 'accept: */*' \
  -H 'content-type: application/json' \
  -d '{
    "name": "string",
    "age": 0,
    "team_id": 0,
    "secret_name": "string"
  }' \
```

Request URL

http://127.0.0.1:8000/heroes/2

Server response

| Code | Details                                                                                                                                                                                                                                |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "name": "string",   "age": 0,   "team_id": 0,   "id": 2 }</pre> <p>Response headers</p> <pre>content-length: 44 content-type: application/json date: Fri, 02 Oct 2026 02:22:37 GMT server: uvicorn</pre> |

Responses

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/heroes/1' \
  -H 'accept: */*' \
```

Request URL

http://127.0.0.1:8000/heroes/1

Server response

| Code | Details                                                                                                               |
|------|-----------------------------------------------------------------------------------------------------------------------|
| 204  | <p>Response headers</p> <pre>content-type: application/json date: Fri, 02 Oct 2026 02:22:59 GMT server: uvicorn</pre> |

Responses

## Question 10:

Khi comment `session.commit()`: Request sẽ bị lỗi hoặc treo/không trả về `id` chuẩn (vì chưa commit nên chưa có ID sinh ra từ DB), và trong Database (`SELECT * FROM hero;`) sẽ **không có** dòng dữ liệu nào được lưu.

Tác dụng của `add()`: Đưa đối tượng vào theo dõi trong phiên làm việc (`session`) của ORM, nhưng chưa ghi dữ liệu chính thức xuống đĩa/Database.

**Tại sao cần `refresh()`:** Sau khi `commit()`, Database tự động tạo các thông tin mới (như `id` khóa chính). Gọi `session.refresh(hero)` giúp nạp lại dữ liệu mới nhất từ Database vào object Python để trả về phản hồi đầy đủ cho client.

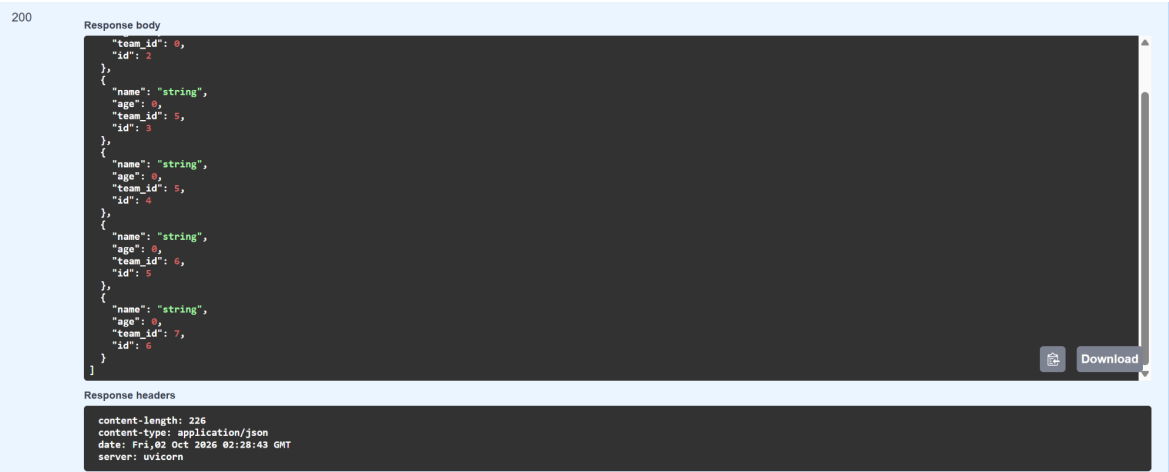
**Question 11:** Nó chỉ cập nhật duy nhất cột `age`. Lý do là nhờ dòng code `hero_in.model_dump(exclude_unset=True)`, biến `extra_data` chỉ chứa những trường thực sự được client gửi lên trong request body chứ không bao gồm các trường bị bỏ trống (`None`).

**Question 12:**

**Trả lời:** Trường `secret_name` không xuất hiện trong dữ liệu JSON trả về.

**Dòng code chịu trách nhiệm:** `response_model=HeroPublic` trong khai báo decorator `@app.get("/heroes/{hero_id}", response_model=HeroPublic)`. Do schema `HeroPublic` không chứa thuộc tính `secret_name` nên FastAPI tự động lọc bỏ nó trước khi gửi phản hồi về cho client.

## 5.2



200

Response body

```
[{"team_id": 0, "id": 1}, {"name": "string", "age": 0, "team_id": 5, "id": 3}, {"name": "string", "age": 0, "team_id": 5, "id": 4}, {"name": "string", "age": 0, "team_id": 6, "id": 5}, {"name": "string", "age": 0, "team_id": 7, "id": 6}]
```

Response headers

```
content-length: 226
content-type: application/json
date: Fri, 02 Oct 2026 02:28:43 GMT
server: uvicorn
```

Responses

## PART 6:

## 6.1

The image contains two screenshots of a web client interface. The top screenshot shows a successful POST request to `http://127.0.0.1:8000/teams` with a JSON body `{ "name": "Avengers", "headquarters": "string" }`. The response is a 201 status code with a JSON body `{ "name": "Avengers", "headquarters": "string", "id": 1 }`. The bottom screenshot shows a failed POST request to the same URL with the same JSON body. The response is a 409 status code with the message `"Team with this name already exists"`. The response headers include `content-length: 47`, `content-type: application/json`, `date: Fri, 02 Oct 2026 02:32:56 GMT`, and `server: uvicorn`.

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/teams' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Avengers",
    "headquarters": "string"
  }'
```

Request URL

`http://127.0.0.1:8000/teams`

Server response

| Code | Details                                                                                                         |
|------|-----------------------------------------------------------------------------------------------------------------|
| 201  | <p>Response body</p> <pre>{   "name": "Avengers",   "headquarters": "string",   "id": 1 }</pre> <p>Download</p> |

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/teams' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Avengers",
    "headquarters": "string"
  }'
```

Request URL

`http://127.0.0.1:8000/teams`

Server response

| Code                               | Details                                                                                                                                                                                                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 409<br><small>Undocumented</small> | <p>Error: Conflict</p> <p>Response body</p> <pre>{   "detail": "Team with this name already exists" }</pre> <p>Download</p> <p>Response headers</p> <pre>content-length: 47 content-type: application/json date: Fri, 02 Oct 2026 02:32:56 GMT server: uvicorn</pre> |

## 6.2

### Question 13:

**Vị trí của 18 và 1:** Các giá trị **18** và **1** không nối trực tiếp vào chuỗi SQL mà được truyền dưới dạng các tham số biến (placeholders/bound parameters như `%(age_1)s`, `%(team_id_1)s`) ở dictionary đi kèm.

**Lý do an toàn chống SQL Injection:** Do Database Engine xử lý câu lệnh SQL và tham số dữ liệu riêng biệt (parameterized queries). Dữ liệu người dùng truyền vào luôn được coi là giá trị đơn thuần, không thể chèn hay làm thay đổi cấu trúc câu lệnh SQL.

### Question 14:

**Hiệu năng & Bộ nhớ (Performance & Memory):** Lọc bằng Python buộc phải tải **toàn bộ dữ liệu** từ Database về bộ nhớ RAM của ứng dụng rồi mới lọc. Khi database có hàng triệu bản ghi, việc này gây cạn kiệt RAM và rất chậm.

**Tối ưu phía DB:** Database được thiết kế chuyên biệt để tìm kiếm và lọc dữ liệu cực nhanh nhờ vào **Indexes**, chỉ gửi lại đúng những bản ghi cần thiết qua mạng.

## 6.3



**Curl**

```
curl -X 'POST' \
  'http://127.0.0.1:8000/teams' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Avengers",
    "headquarters": "string"
  }'
```

**Request URL**

http://127.0.0.1:8000/teams

**Server response**

| Code | Details         |
|------|-----------------|
| 409  | Error: Conflict |

*Undocumented*

**Response body**

```
{
  "detail": "Team with this name already exists"
}
```

**Response headers**

```
content-length: 47
content-type: application/json
date: Fri, 02 Oct 2026 02:46:44 GMT
server: uvicorn
```

**Request URL**

http://127.0.0.1:8000/heroes

**Server response**

| Code | Details          |
|------|------------------|
| 404  | Error: Not Found |

*Undocumented*

**Response body**

```
{
  "detail": "Team not found"
}
```

**Response headers**

```
content-length: 27
content-type: application/json
date: Fri, 02 Oct 2026 02:47:23 GMT
server: uvicorn

Responses


```

## PART 7:

**Question 15:** Quy tắc của `create_all()` là kiểm tra bảng nào **chưa tồn tại** trong Database thì mới thực hiện câu lệnh `CREATE TABLE` để tạo mới. Vì bảng `hero` đã được tạo từ trước đó rồi nên nó sẽ bỏ qua, còn hai bảng `mission` và `heromissionlink` là các model mới chưa có dưới Database nên sẽ được tự động tạo.

## PART 8:

Parameters Cancel

| Name                                                      | Description                    |
|-----------------------------------------------------------|--------------------------------|
| <b>hero_id</b> <span>required</span><br>integer<br>(path) | <input type="text" value="1"/> |

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/heroes/1/missions' \
-H 'accept: */*'

```

Request URL

```
http://127.0.0.1:8000/heroes/1/missions

```

Server response

| Code                       | Details                                           |
|----------------------------|---------------------------------------------------|
| 404<br><i>Undocumented</i> | Error: Not Found<br>Response body<br><pre>{</pre> |

```

app > seed.py > ...
5  def seed_db():
63  .....team_avengers,team_xmen,
64  .....mission_sokovia,mission_endgame,
65  .....hero1,hero2,hero3,hero4,hero5
66  .....]
67  .....session.commit()
68  .....print("Database seeded successfully!")
69
70  if __name__ == "__main__":
71  .....seed_db()

```

PROBLEMS 71 OUTPUT DEBUG CONSOLE PORTS SPELL CHECKER 71

TERMINAL

```

2026-10-02 09:59:54,709 INFO sqlalchemy.engine.Engine [raw sql] ()
2026-10-02 09:59:54,710 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("team")
2026-10-02 09:59:54,711 INFO sqlalchemy.engine.Engine [raw sql] ()
2026-10-02 09:59:54,711 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("hero")
2026-10-02 09:59:54,711 INFO sqlalchemy.engine.Engine [raw sql] ()
2026-10-02 09:59:54,712 INFO sqlalchemy.engine.Engine PRAGMA main.table_info("mission")
2026-10-02 09:59:54,713 INFO sqlalchemy.engine.Engine [raw sql] ()
2026-10-02 09:59:54,714 INFO sqlalchemy.engine.Engine COMMIT
2026-10-02 09:59:54,715 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-10-02 09:59:54,731 INFO sqlalchemy.engine.Engine SELECT team.name, team.headquarters, team.id
FROM team
2026-10-02 09:59:54,731 INFO sqlalchemy.engine.Engine [generated in 0.00069s] ()
Database already seeded! Skipping.
2026-10-02 09:59:54,735 INFO sqlalchemy.engine.Engine ROLLBACK
(.venv) PS C:\Users\Meo Beo\OneDrive - National Economics University\HocWeb\hero-api>

```

GET /teams/{team\_id}/heroes Read Team Heroes

Parameters

| Name               | Description |
|--------------------|-------------|
| team_id * required |             |
| integer            | 1           |
| (path)             |             |

Execute Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/teams/1/heroes' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/teams/1/heroes
```

Server response

| Code | Details       |
|------|---------------|
| 200  | Response body |

## Question 16:

### Thứ tự thực thi INSERT:

1. SQLAlchemy sẽ thực thi `INSERT INTO team ...` và `INSERT INTO mission ...` trước.
2. Tiếp theo, nó chạy `INSERT INTO hero ....`
3. Cùng lúc đó, nó chạy các lệnh `INSERT INTO heromissionlink ...` cho bảng liên kết N-N.

**Cách hero.team\_id lấy giá trị:** Do chúng ta truyền trực tiếp đối tượng `team=team_avengers` vào hero, khi SQLAlchemy chèn bản ghi `Team` vào DB, nó sử dụng cơ chế `RETURNING id` để lấy `id` vừa được tự động sinh ra từ cơ sở dữ liệu. Sau đó, SQLAlchemy tự động gán giá trị `id` đó vào thuộc tính `team_id` của `Hero` trước khi phát lệnh `INSERT INTO hero`.